

Structuri de Date

Anul universitar 2019-2020
Prof. Adina Magda Florea



Curs Nr. 12

Metode de sortare

- ☐ Metode elementare
- ☐ Quicksort
- ☐ Mergesort



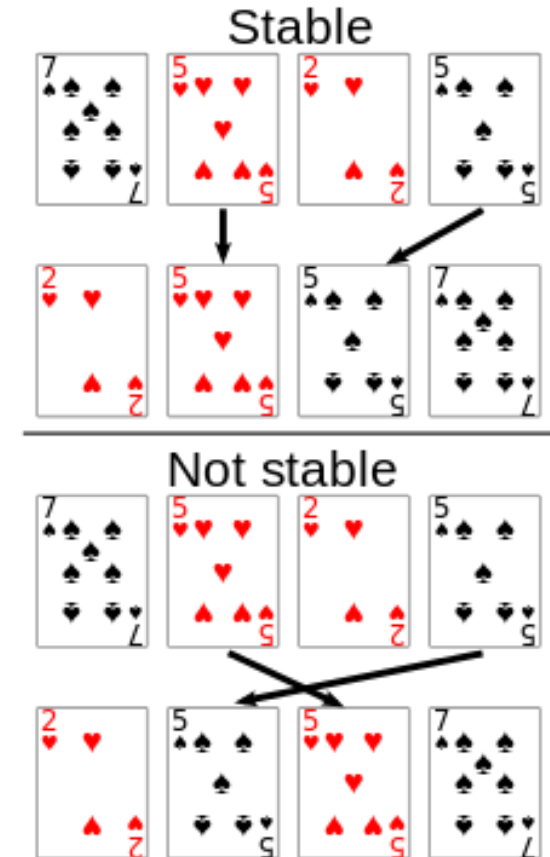
1. Metode de sortare

- **Metode elementare** – potrivite pentru un număr mic de elemente (tipic $< 500-1000$)
- **Metode avansate de sortare** – potrivite pentru fișiere mari
- De obicei metodele elementare $O(N^2)$
- iar cele avansate $O(N \log N)$
- **Sortare internă**
- **Sortare externă**
- Criterii:
 - timpul de rulare
 - cantitatea de memorie suplimentară



Metode de sortare

- **Metoda de sortare stabilă** – menține ordinea relativă a înregistrărilor cu chei egale
- Dacă stabilitatea este importantă se poate crea o nouă cheie – cheie extinsă
- **Sortare indirectă** - dacă înregistrările sunt mari este bine să nu le schimbăm poziția
- Cei mai mulți algoritmi folosesc o santinelă (celulă suplimentară)



2. Metode de sortare elementare

Sortarea prin Selecție (Selection Sort)

- Sortam $a[1]..a[N]$
 - găsește cel mai mic element și îl schimbă cu elementul de pe prima poziție, etc.
 - pentru fiecare $i = 1, N-1$ schimbă $a[i]$ cu elementul minim din $a[i+1]..a[N]$
- Deoarece fiecare element este mutat cel mult o dată, este o metodă bună pentru a sorta secvențe cu ***înregistrări mari și chei mici***

Sortare prin Selecție

```
void Selection(int a[], int N)
{
    int i, j, min, t;
    for(i=1; i < N; i++)
    {
        min = i;
        for(j=i+1; j<=N; j++)
            if (a[j] < a[min]) min = j;
        t = a[min]; a[min] = a[i]; a[i] = t;
    }
}
```

Sortare prin inserție

Sortarea prin Inserție (**Insertion Sort**)

- Consideră fiecare element pe rând, inserând elementul la locul potrivit printre cele deja sortate
- pentru fiecare $i = 2, N$ elementele $a[1] \dots a[i]$ sunt sortate punând $a[i]$ pe poziția corectă între elementele deja sortate $a[1] \dots a[i-1]$
- Programul următor folosește o *santinelă* – în $a[0]$ este pus un întreg mai mic decât orice element din vector
- Dacă nu folosim santinelă atunci adăugăm testul pentru $j, j > 1$.

Sortare prin Inserție

```
void Insertion(int a[], int N)
{
    int i, j, v;
    for(i=2; i<=N; i++)
    {
        v = a[i];  j = i;
        while( a[j-1] > v )
            { a[j] = a[j-1]; j--; }
        a[j] = v;
    }
}

a[0]= INT_MIN;
```


Sortare prin metoda bulelor

```
void Bubble(int a[], int N)
{  int i, j, t;
   for(i = N; i >= 1; i--)
       for(j = 2; j <= i; j++)
           if( a[j-1] > a[j] )
               { t=a[j-1]; a[j-1]=a[j]; a[j]=t; }
}
```

Performanța metodelor elementare

- Sortarea prin **selecție** folosește $\sim N^2/2$ comparații și **N interschimbări**
- Sortarea prin **insertie** folosește $\sim N^2/4$ **comparații** și **$N^2/8$ interschimbări** în medie, și dublu în cazul cel mai nefavorabil
- Sortarea prin insertie este *liniară* pentru fișiere “aproape sortate”.
- Sortarea prin **metoda bulelor** folosește $\sim N^2/2$ **comparații** și **$N^2/2$ interschimbări** pentru cazul mediu și pentru cazul cel mai nefavorabil
- Toate metodele au **$O(N^2)$** cazul cel mai nefavorabil

Algoritm	Timp			Spațiu
	Cel mai bun (Best)	Mediu (Average)	Cel mai rău (Worst)	
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$

Sortare indirectă

- Înregistrări de dimensiune mare
- Folosim un vector de indici **p**: $p[1]..p[N]$
- Algoritmii trebuie modificați pentru a se referi la **a[p[i]]** în loc de **a[i]** când se compara **a[i]** și să se refere la **p** în loc de **a** când se face sortarea
- După ce s-a realizat sortarea indirectă, dacă se dorește se pot rearanja înregistrările
- Se poate utiliza și un vector de pointeri

Sortare prin Inserție - indirectă

```
void Insertion(int a[], int p[], int N)
{
    int i, j, v;
    for(i=1; i<=N; i++) p[i] = i;
    for(i=2; i<=N; i++)
    {
        v = p[i];  j = i;
        while( a[p[j-1]] > a[v] )
            { p[j] = p[j-1]; j--; }
        p[j] = v;
    }
}
```

```
a[0]= INT_MIN; /* santinela */
```

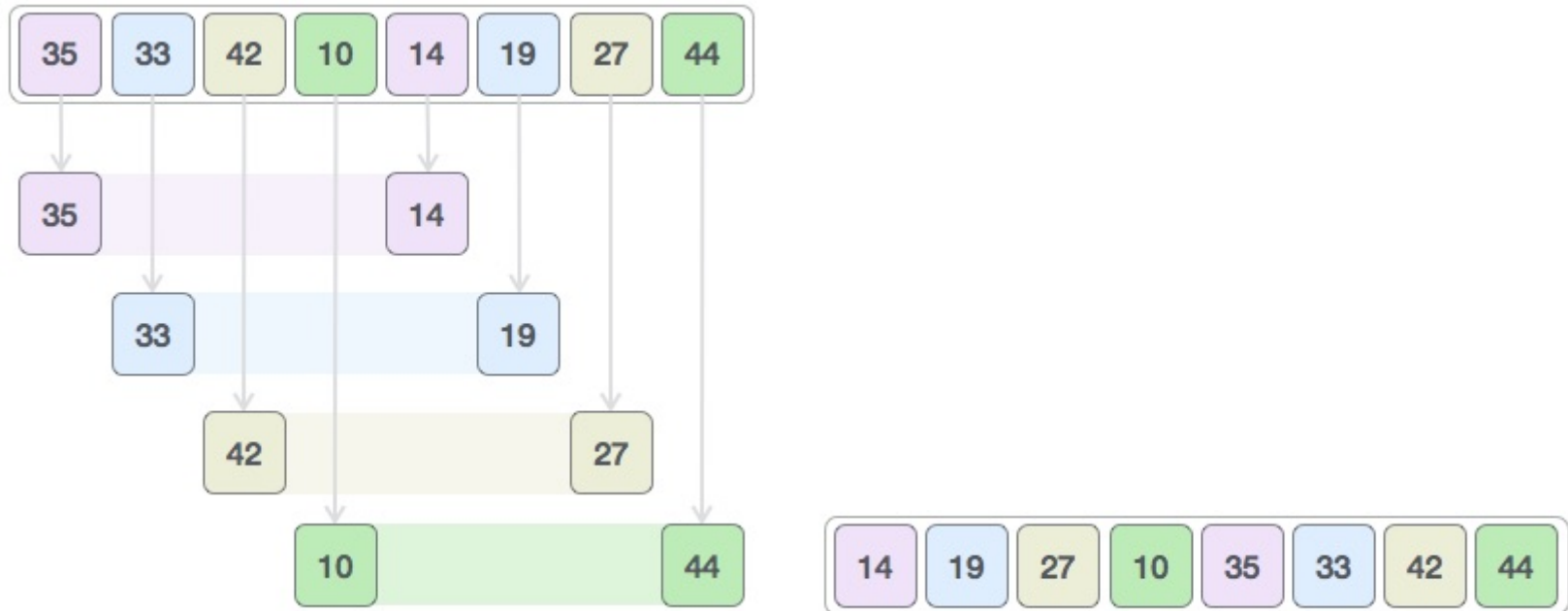


Sortare Shell

Sortarea Shell (**Shell Sort**)

- Extindere a Insertion Sort – crește viteza permițând interschimbări între elemente neadiacente
- Rearanjăm vectorul de sortat a.î. să aibă proprietatea:
 - Dacă se ia fiecare al ***h-lea*** element (începând de oriunde) obținem un vector sortat.
- Un astfel de vector se numește **h-sortat** iar **h** se numeste in engleza **gap**
- Un vector h-sortat poate fi văzut ca o colecție de **h** vectori independenți întrețesuți

Shell Sort



- Pentru fiecare h folosim Insertion Sort, independent pentru fiecare din cele h subșiruri
- Nu putem folosi santinele în acest caz deoarece ar trebui să avem h santinele

Shell Sort

- Există multe variante de alegere a valorilor h
- Formula lui Knuth **$h=1$ și $h=h*3+1$**
- Fie $h_1 = 1$, $h_{s+1} = 3h_s + 1$, și ne oprim cu h_t pentru care $h_{t+2} \geq N$

Exemplu

$$h_1 = 1$$

$$h_2 = (3 \times 1) + 1 = 4$$

$$h_3 = (3 \times 4) + 1 = 13$$

$$h_4 = (3 \times 13) + 1 = 40$$

$$h_5 = (3 \times 40) + 1 = 121$$

$$N=100 - \text{selectez } h_5$$

Valoarea finala (h_t) este h_3 .

Secventa valorilor lui h este 13-4-1.

Valorile lui h pornind de la valoarea initiala se calculeaza:

$$h_{s-1} = [h_s / 3]$$

Shell Sort

```
void Shellsort(int a[], int N)
{  int i, j, h, v;
    for(h = 1; h <= N; h = 3*h+1);
    h /= 3; h /= 3;
    for( ; h > 0; h /= 3)
        for(i = h+1; i <= N; i++)
            { v = a[i];  j = i;
              while( j>h && a[j-h] > v )
                  { a[j] = a[j-h]; j -= h; }
              a[j] = v;
            }
}
```

- Secvența mai mare ...1093, 364, 121, 40, 13, 4, 1

3. Metode de sortare avansate

- **Heapsort** (vezi curs 8)
 - **Quicksort**
 - **Mergesort**
-
- **Sortare topologică** (vezi curs 10)

Quicksort

- Probabil cel mai utilizat algoritm de sortare
- 1960 – C.A.R. Hoare
- Consumă resurse mai puține decât alte metode
- In **medie** folosește **$N \log N$** operații pentru a sorta N elemente dar **N^2** pentru cazul cel mai nefavorabil

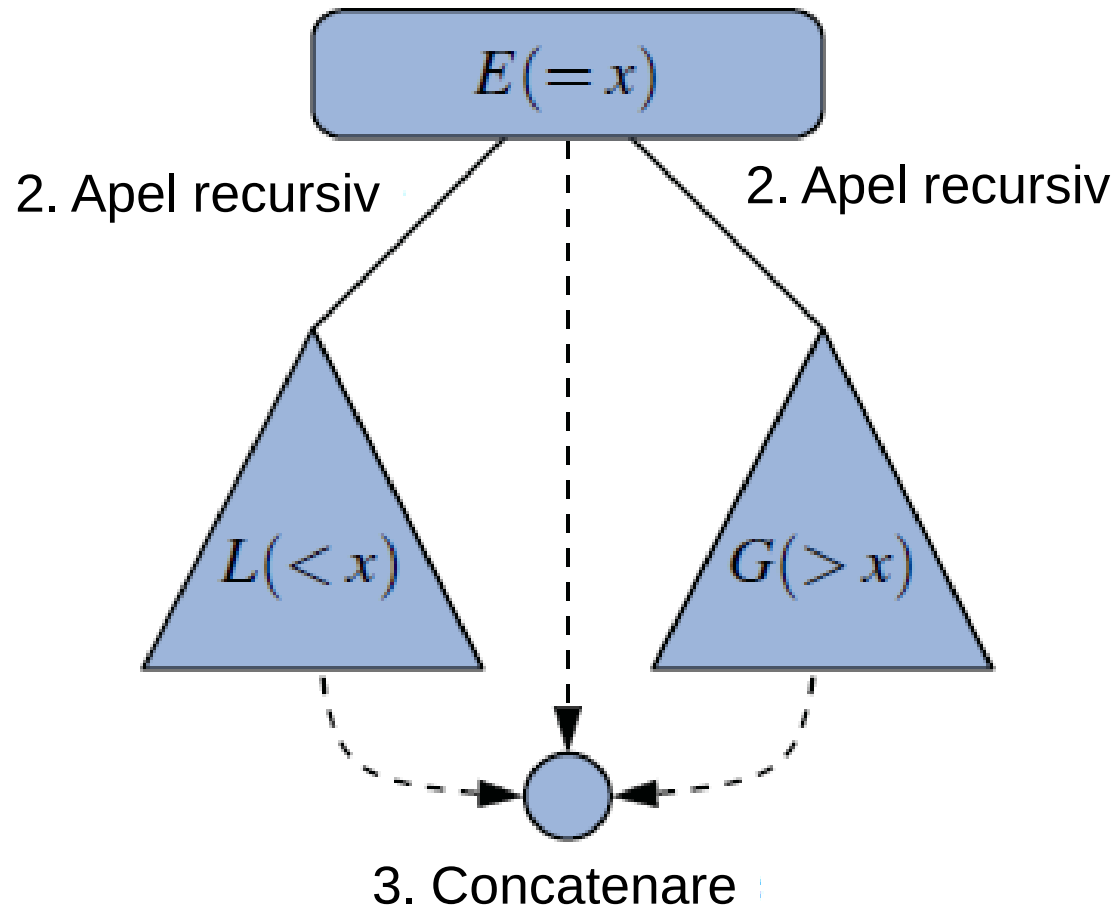
C.A.R. Hoare, englez (1934)

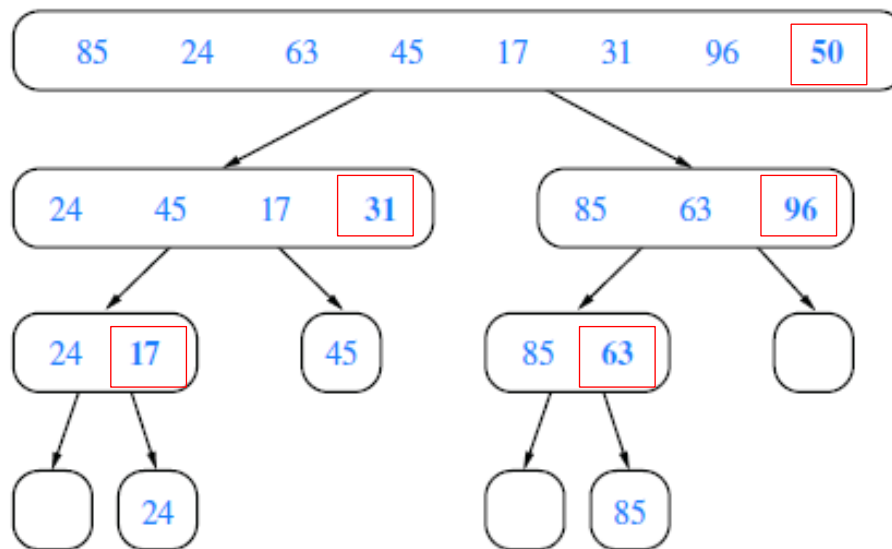


Quicksort

- Folosește paradigma “**divide-and-conquer**”
- **Divide:** dacă S (șirul de sortat) are cel puțin două elemente, selectează un element x din S – **pivot** (tipic ultimul). Împarte elementele din S în 3 părți:
 - Elemente mai mici decât x (L)
 - Elemente egale cu x (E)
 - Elemente mai mari ca x (G)
- **Conquer:** sortează L și G
- **Combine:** Reface S punând în S , în ordine: L , E și G

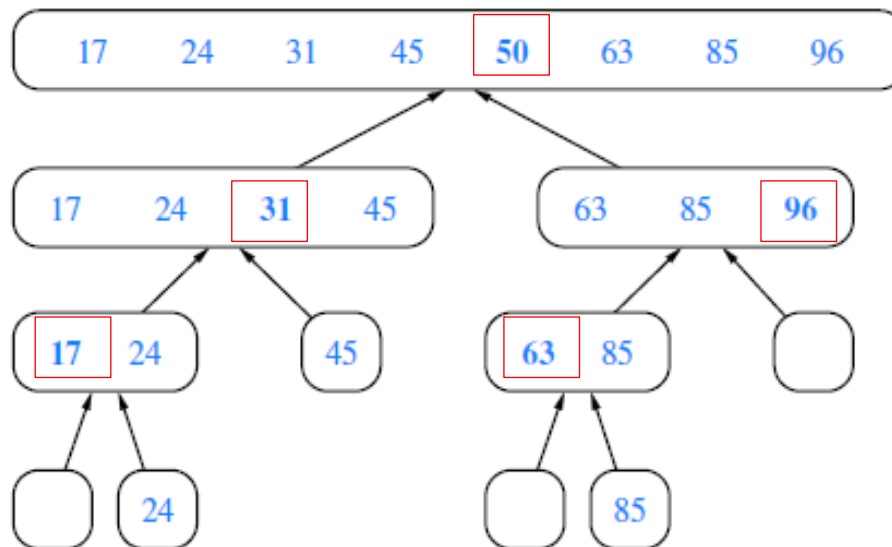
1. Imparte utilizand pivotul x





(a)

Arborele Quicksort
pentru algoritmul aplicat
pe o secventa de 8 elemente



(b)

QuickSort

```
void Quicksort(int a[], int l, int r)
{   int i;
    if (r > l)
    {   i = partition(a, l, r);
        quicksort(a, l, i-1);
        quicksort(a, i+1, r);
    }
```

Quicksort(a, 1, N) sorteaza toata secventa



QuickSort

```
void Quicksort(int a[], int l, int r)
{   int v, i, j, t;
    if (r > l)
    {   v = a[r]; i = l-1; j = r;
        for( ; ; )
        {   while (a[++i] < v);
            while (a[--j] > v);
            if (i >= j) break;
            t=a[i]; a[i]=a[j]; a[j]=t;
        }
        t=a[i]; a[i]=a[r]; a[r]=t;
        quicksort(a, l, i-1);
        quicksort(a, i+1, r);
    }
}
```

} partition(a,l,r)

a[0]= INT_MIN; /* santinela */



Quicksort – performanțe

- Limitare – ineficient dacă secvența este deja sortată
- Folosește aproximativ **$2N \ln N$** comparații pentru cazul mediu
- **$O(N^2)$** cazul cel mai defavorabil
- **$O(N \log N)$** cazul mediu
- Folosește spațiu suplimentar în stivă



QuickSort nerecursiv

```
void Quicksort(int a[], int N)
{   int i, l, r;
    l=1; r=N; initstack(s);
    for( ; ; )
    {   while (r > l)
        {   i = partition(a, l, r);
            if (i-l > r-i)
                { push(s, l); push(s, i-1);
                  l = i+1; }
            else
                { push(s, i+1); push(s, r);
                  r = i-1;   }
        }
        if (stackempty(s)) break;
        r = pop(s); l = pop(s);
    }
}
```

Mergesort

- Folosește paradigma **“divide-and-conquer”**
- Poate fi implementat a.i. să acceseze elementele într-o manieră secvențială
- Avantaje: potrivit pentru sortarea secvențelor reprezentate prin liste înlănțuite
- Potrivit ca bază pentru extindere la sortări externe
- Two-way merging
- Multi-way merging

Mergesort

- Folosește paradigma **“divide-and-conquer”**
- **Divide:** dacă S are 0 sau 1 element, întoarce soluția. Altfel (S are cel puțin 2 elemente) împarte S în 2 părți disjuncte aproximativ egale:
 - S_1 cu primele $N/2$ elemente
 - S_2 cu ultimele $N/2$
- **Conquer:** sortează S_1 și S_2
- **Combine:** Reface S prin combinarea (îmbinarea) lui S_1 și S_2 într-o secvență sortată = **merge**

2-way Merge (imbinare dubla)



$a[1] \geq b[1]$, incrementeaza i

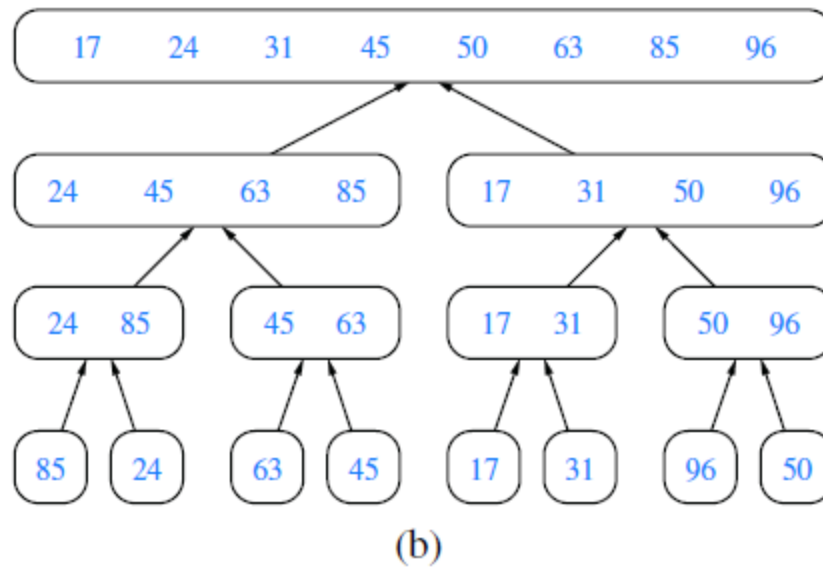
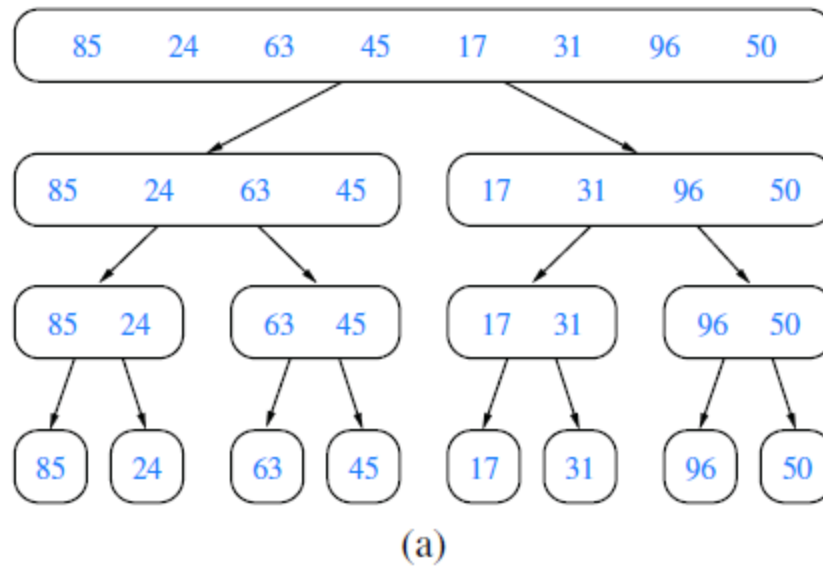


$a[1] < b[1]$, incrementeaza j



$c[k]$

Arborele mergesort



Mergesort

Presupunem $a[1] \dots a[M]$ sortat si $b[1] \dots b[N]$ sortat

```
i=1; j=1;
```

```
a[M+1] = INT_MAX; /* santinela */
```

```
b[N+1] = INT_MAX; /* santinela */
```

```
for(k=1; k<=M+N; k++)
```

```
    c[k] = (a[i] < b[j]) ? a[i++] : b[j++]
```

Mergesort

```
void mergesort(int a[], int l, int r)
{ int i,j,k,m;
  if(r > l)
  { m = (r+l)/2;
    mergesort(a, l,m);
    mergesort(a,m+1,r);
    for(i = m+1; i>l; i--) /* 1 */
      b[i-1] = a[i-1];
    for(j = m; j<r; j++) /* 2 */
      b[r+m-j] = a[j+1];
    for(k = l; k<=r; k++) /* 3 */
      a[k] = (b[i] < b[j]) ? b[i++] : b[j--];
  }
} /* nu are nevoie de santinela */
```


Mergesort pe listă

```
typedef int Item;
typedef struct cel {
    Item    elem;
    struct  cel  *next;} ListCel, *TList;
Tlist z; /* foloseste o celula santinela z */
/* la initializare adaug z->elem = INT_MAX */
Tlist merge(Tlist a, Tlist b)
{ Tlist c;
  c = z;
  do {
      if (a->elem <= b->elem)
          { c->next = a; c = a; a = a-> next; }
      else
          { c->next = b;  c = b; b = b->next; }
  } while (c != z);
  c = z->next; z->next = z;
  return c;
}
```

Mergesort pe listă

```
Tlist mergesort(Tlist c)
{ Tlist a, b;
  if (c->next != z)
  { a = c; b = c->next->next->next;
    while (b != z)
    { c = c->next; b = b->next->next; }
    b = c->next; c->next = z;
    return merge(mergesort(a), mergesort(b));
  }
  return c;
}
```

Se poate folosi si lungimea listelor

Mergesort – performanță

- Necesită aproximativ **$N \log N$ comparații** pentru a sorta un sir de N elemente
- Folosește un **spațiu proporțional cu N**
- Este o metodă **stabilă**
- **$O(N \log N)$** pentru cazul mediu și cazul cel mai defavorabil

Performanța algoritmilor de sortare

Algoritm	Timp			Spațiu
	Cel mai bun (Best)	Mediu (Average)	Cel mai rău (Worst)	
Quicksort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$
Mergesort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Heapsort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Shell Sort	$O(N)$	$O((N \log N)^2)$	$O((N \log N)^2)$	$O(1)$
Radix Sort	$O(Nk)$	$O(Nk)$	$O(Nk)$	$O(N+k)$

Sortare fara comparare

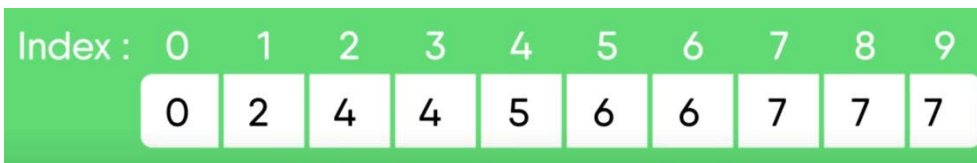
- Metodele studiate pana acum sunt metode de sortare bazate pe compararea elementelor
- Pentru timpul mediu si timpul cel mai defavorabil nu putem obtine o valoare a complexitatii timp sub **$O(N\log N)$**
- Exista algoritmi de sortare care nu implica compararea elementelor si care au timp mediu si defavorabil proportional cu N dar care se pot aplica numai pentru anumite categorii de date
- Sortarea prin numarare
- Sortarea Radix

Sortarea prin numarare

- Este un algoritm de sortare care se aplica daca cheile sunt intr-un anumit interval
- Funcționează prin numărarea numărului de obiecte care au valori de chei distincte (un fel de hashing)
- Apoi realizeaza calcule aritmetice pentru a calcula poziția fiecărui obiect din secvența de ieșire.

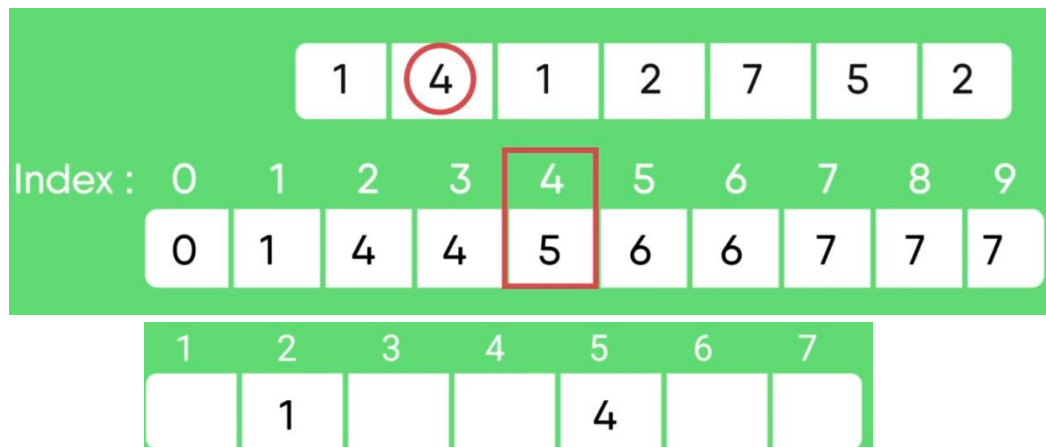
Sortarea prin numarare

- Presupunem cheile in intervalul $[0,9]$
- Folosim un vector index cu un numar de valori egal cu numarul de elemente din interval, initial 0



Sortarea prin numarare

- Scriem la iesire fiecare element, dupa care decrementam contorul



Sortarea prin numarare

- Scriem la iesire fiecare element, dupa care decrementam contorul

			1	4	1	2	7	5	2	
Index :	0	1	2	3	4	5	6	7	8	9
	0	1	4	4	5	6	6	7	7	7

1	2	3	4	5	6	7
	1			4		

			1	4	1	2	7	5	2	
Index :	0	1	2	3	4	5	6	7	8	9
	0	0	3	4	4	5	6	6	7	7

1	2	3	4	5	6	7
1	1	2	2	4	5	7

```

void countSort(int a[])
{
    int output[SIZE];
    /* dimensiunea vectorului de sortat */
    int count[RANGE + 1], i;
    /* domeniul de valori al elementelor */
    for(i = 1; i <= RANGE; ++i) count[i]=0;
    /* calculeaza nr de aparitii */
    for(i = 0; SIZE-1; ++i) ++count[a[i]];
    for (i = 1; i <= RANGE; ++i)
        count[i] += count[i-1];
    for (i = 0; a[i]; ++i)
    {
        output[count[a[i]]-1] = a[i];
        --count[a[i]];
    }
    for (i = 0; a[i]; ++i)
        a[i] = output[i];
}

```

Sortarea prin numarare

- Pentru a fi stabila inlocuim

```
for (i = 0; SIZE-1; ++i)
{
    output[count[a[i]]-1] = a[i];
    --count[a[i]];
}
```

Cu

```
for (i = SIZE-1; i>=0; --i)
{
    output[count[a[i]]-1] = a[i];
    --count[a[i]];
}
```

- Are complexitatea **$O(N + K)$** daca sunt **N** elemente de sortat aflate in intervalul **1..K**

Sortarea Radix

- Daca elemente sunt in intervalul $1..N^2$, nu putem folosi sortarea prin numarare deoarece sortarea aceasta ar avea un timp de $O(N^2)$.
- Putem folosi **Radix Sort** – numai in cazul in care avem elemente numere intregi
- Ideea este de a face o sortare dupa fiecare cifra începând de la cea mai putin semnificativa pana la cea mai semnificativa.
- Sortarea Radix folosește sortarea prin numarare ca subrutină pentru a sorta.

Sortarea Radix

- Sorteaza elementele folosind sortarea prin numarare (sau alt algoritm de sortare stabil) pentru fiecare digit de pe pozitia i , pornind de la cel mai putin semnificativ

170	45	75	90	802	24	2	66
-----	----	----	----	-----	----	---	----

170	45	75	90	802	24	2	66
-----	----	----	----	-----	----	---	----

170	90	802	2	24	45	75	66
-----	----	-----	---	----	----	----	----

170	90	802	2	24	45	75	66
802	2	24	45	66	170	75	90
2	24	45	66	75	90	170	802

```

/* sortare prin numarare a[] in fct de digit exp */
void countSort(int a[], int n, int exp)
{ int output[n]; int i, /* init cu zero count[10] */
  /* numara nr aparitii in count */
  for (i = 0; i < n; i++) count[ (a[i]/exp)%10 ]++;
  /* modifica count[i] a.i. sa contina pozitia
    curenta a acestui digit in output[] */
  for (i = 1; i < 10; i++) count[i] += count[i - 1];
  /* construiesc vectorul de iesire */
  for (i = n - 1; i >= 0; i--)
  {   output[count[ (a[i]/exp)%10 ] - 1] = a[i];
      count[ (a[i]/exp)%10 ]--;
  }
  for (i = 0; i < n; i++)
      a[i] = output[i];
}

```

```
int getMax(int a[], int n)
{
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max) max = a[i];
    return max;
}
```

```
void radixsort(int a[], int n)
{
    /* gaseste nr maxim pt a afla cati digiti */
    int m = getMax(arr, n);
    /* sortarea prin numarare pt fiecare digit */
    /* exp este 10^i unde i este digitul curent */

    for (int exp = 1; m/exp > 0; exp *= 10)
        countSort(a, n, exp);
}
```

Discutie

- Daca elementele de sortat au d digiti complexitatea timp este $O(d \cdot (N + b))$ în care b este baza de reprezentare a numerelor, de exemplu, pentru sistemul zecimal, b este 10.
- Care este valoarea lui d ? Dacă k este valoarea maximă posibilă, atunci d este $O(\log_b(k))$. Deci complexitatea timpului general este $O((N + b) \cdot \log_b(k))$.
- Aceasta este mai mare decât complexitatea în timp a algoritmilor de sortare bazati pe comparație pentru un k mare.
- Putem limita k . Fie $k \leq N^c$ unde c este o constantă.
- În acest caz, complexitatea devine $O(N \log_b(N))$.

Discutie

- Dar complexitatea este inca mai mare decat cea a algoritmilor de sortare bazati pe comparație.
- Care ar trebui să fie valoarea lui **b** pentru ca sa avem o complexitate timp liniara?
- Dacă setăm **b** la **N**, obținem complexitatea timp devine **$O(N)$** .
- Deci putem sorta o serie de numere întregi aflate in intervalul **$1..N^c$** dacă numerele sunt reprezentate în baza **N** (sau fiecare cifră necesita $\log_2(N)$ biți).

Discutie

- Este de preferat sortarea Radix comparativ cu algoritmi de sortare pe bază de comparație precum sortarea rapidă?
- Dacă avem $\log_2 N$ biți pentru fiecare cifră, complexitatea timp a Radix Sort pare a fi mai bună decât cea a Quicksort pentru o gamă largă de numere de intrare.
- Factorii constanți ascunși în notația asimptotică sunt mai mari pentru Radix Sort și Quicksort folosește mai eficient cache-urile hardware.
- În plus, sortarea Radix folosește sortarea prin numărare ca subrutină, iar aceasta necesită spațiu suplimentar pentru a sorta numere.

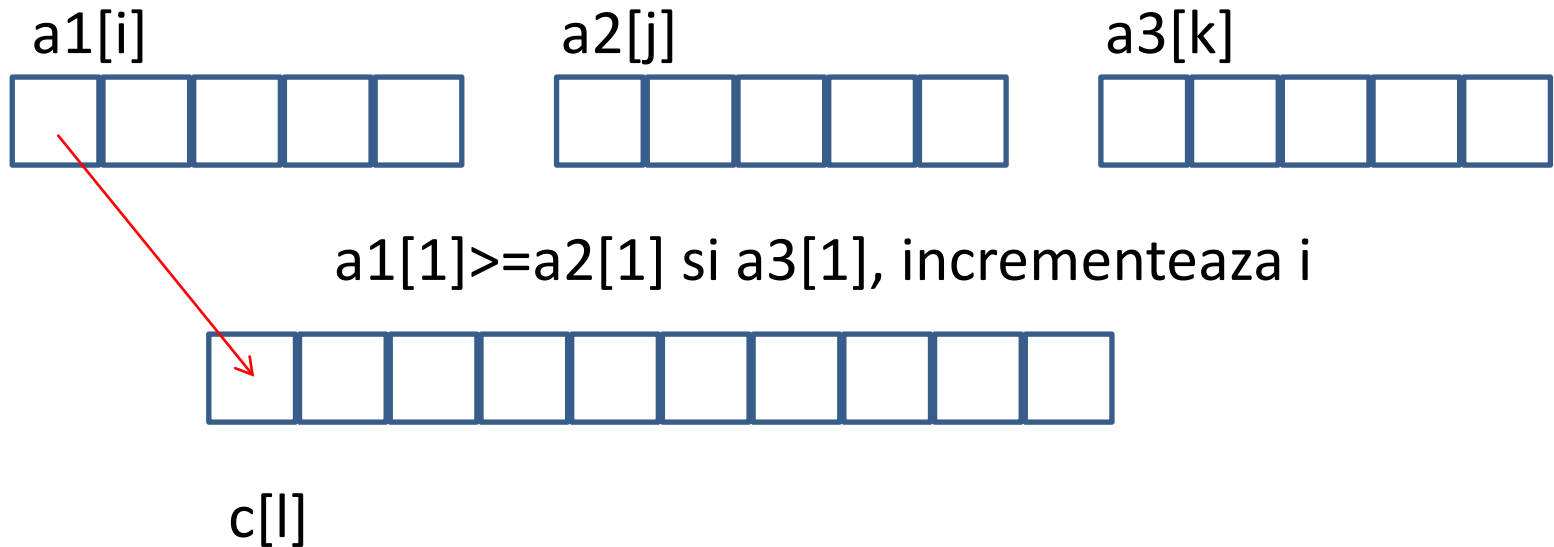
Algoritmi de sortare externa

- Sortarea masivelor mari de date care nu pot fi incarcate simultan in memorie
- Algoritmii de sortare externă se încadrează, în general, în două categorii:
 - Merge sort extern (adaptare Merge Sort)
 - Sortare prin distributie (adaptare Quicksort)

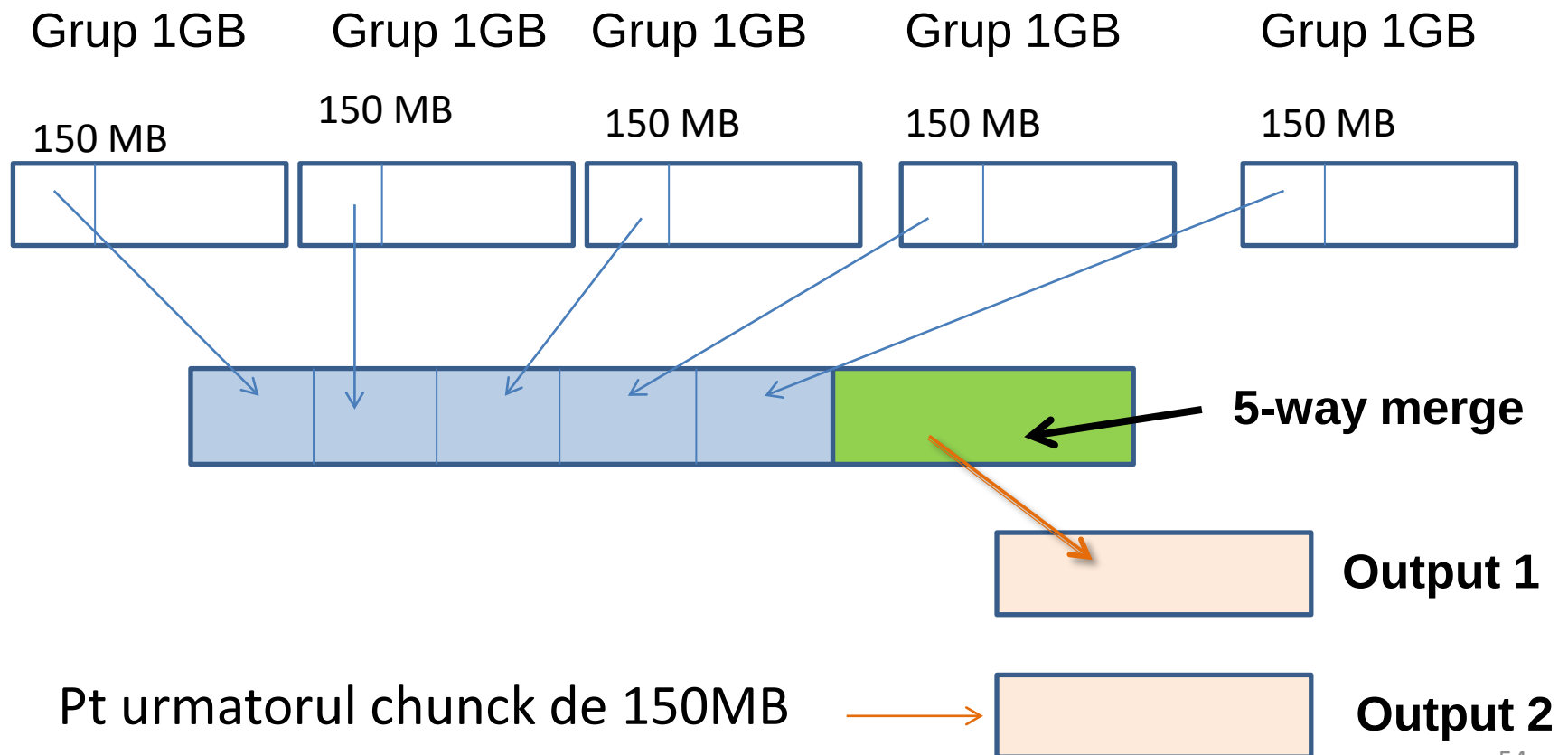
Merge Sort Extern

- **Merge Sort pentru sortare externa este un algoritm cu K imbinari / intreteseri (K-way merge)**
- Sorteaza grupuri de elemente care pot intra in memorie si apoi combina aceste grupuri
- Sortează întâi grupuri (chunks) de cate M elemente in memoria interna (dintre cele N elemente externe)
- Pune grupurile sortate inapoi pe memoria externa
- Se face o combinatie cu M/B imbinari a acestor liste sortate
- Pentru a face aceasta combinatie, se incarca in memoria interna B elemente din fiecare grup de elemente sortate, executa k-way merge si scrie elementele la iesire, in mod repetat

k-way Merge (k=3)



- 1 GB RAM pt a sorta 5GB de date
- Imparte 5GB in 5 grupuri
- Sorteaza pe rand fiecare grup in memorie cu un algorithm de sortare (interna) si scrie-l in memoria externa



Merge Sort sortare externa

- Complexitate **$O(N \log N)$**
- Limitare la combinare într-o singura trecere : pe măsură ce numărul de grupuri (chunks) crește, memoria va fi împărțită în mai multe buffere, astfel încât fiecare buffer este mai mic
- Acest lucru determină mai multe citiri din memoria externa

Merge Sort sortare externa

- De exemplu, pentru a sorta, 50 GB în 100 MB RAM, folosind o singură trecere de combinare, se face completarea bufferelor de intrare cu date de la fiecare dintre cele 500 de grupuri (citim 100MB / 501 ~ 200KB de la fiecare grup la un moment dat) a.i. citirea ocupă cea mai mare parte a timpului de sortare. Utilizarea a două treceri de combinare (2-path k-way merge) rezolvă problema.

Cum se poate face

- Se executa sortarea grupurilor ca inainte
- Se executa o prima trecere de combinare in care se combina cate 25 de grupuri o data, rezultand 20 de grupuri sortate mai mari
- Se executa o a doua trecere pentru a combina cele 20 de grupuri sortate mai mari si se obtin datele sortate

Animații pt. algoritmi de sortare

- <http://www.sorting-algorithms.com/>
- <https://www.youtube.com/watch?v=XaqR3GNVoo>
- <https://www.youtube.com/watch?v=ZZuD6iUe3Pc>

Know Thy Complexities!

- <http://bigocheatsheet.com/>